IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2026.0000000

Frontier and Localhost: A Survey of How Production AI Learns Outside the Weights

MIKHAIL L. ARBUZOV[1], LEE MOSBACKER[2], SISONG BEI[3], ZIWEI DONG[4], DMITRI KALAEV[5], AND ALEXEY SHVETS[6]

[1]Independent Researcher, CITY, COUNTRY (e-mail: mike.arbuzov54@gmail.com)
[2]Independent Researcher, CITY, COUNTRY (e-mail: lee.mosbacker@gmail.com)
[3]Independent Researcher, CITY, COUNTRY (e-mail: qurining@gmail.com)
[4]Independent Researcher, CITY, COUNTRY (e-mail: ziwei.dong@alumni.emory.edu)
[5]Independent Researcher, CITY, COUNTRY (e-mail: kalaevdr@gmail.com)
[6]Palo Alto Networks, CITY, COUNTRY (e-mail: ashvets@paloaltonetworks.com)

Corresponding author: Mikhail L. Arbuzov (e-mail: mike.arbuzov54@gmail.com).

TODO(authors): funding statement, e.g., "This work received no external funding." A. Shvets is employed by Palo Alto Networks; the views expressed are the authors' own.

ABSTRACT Production systems built on large language models increasingly adapt outside the model weights: after a failure, teams change instructions, skills, memories, tools, orchestration graphs, evaluation suites, and governance rules faster than any weight update. Two earlier results explain why this is the right place to adapt: reliability over long outputs hinges on a sparse set of key decisions, and inside a bounded deployment those decisions fail in a small, recurring catalogue of modes that a targeted budget of fixes can cover. This article surveys 256 production and research systems from January 2023 to October 2026 and reads the scaffold layer as one adaptation surface governed by one loop: a recurring failure is traced to a scaffold coordinate, a candidate edit is proposed, a gate accepts or rejects it on measured loss, and a second decision sets how far the edit may travel. We state the five conditions under which this loop is an optimizer rather than patchwork, classify systems by which loops they close and whether each gate is algorithmic or human, and add a further axis: the class of feedback signal the gate receives. Exact signals (code, mathematics), observable outcomes (completed tasks), judged signals (writing, design), and delayed signals (research) fix what the gate can be, which design-space cells are reachable, and which failure dominates. No surveyed system combines automated local adaptation with governed cross-organizational promotion, versioned lineage, access control, and rollback. We map the failure surface this gap leaves open and state the open problems for closing it.

INDEX TERMS AI governance, agent memory, agent skills, evaluation, large language models, LLM agents, prompt optimization, self-improving systems, survey

I. INTRODUCTION

The standard mental model of a system built on a large language model (LLM) is the model. Pre-training fixes the weights, fine-tuning nudges them, and deployment wraps a thin layer of prompts and tools around the result. Production in 2025 and 2026 has stopped behaving this way. Frontier weights change on a cycle of months. The systems built on them change daily, in instruction files checked into repositories, in memories that persist across sessions, in skills loaded on demand, in registered tools, in evaluation suites that block a merge, in agent topologies, and in pull-request gates on shared rule repositories. A growing share of the adaptation that makes a deployed system reliable now happens outside the weights, in a fast-changing local scaffold wrapped around a slow-changing frontier model.

Read one at a time, these artifacts look like unrelated engineering. A skill file is a procedure the model did not have. A repository rule is a local parameter for one project's conventions. A retrieval connector is memory access specific to one patch of work. A tool registry provisions a capability the base model cannot reach. A pull-request review against a shared rule repository is a promotion gate. An evaluation suite that blocks a merge is a validation check before promotion. Read together, they are one adaptation surface, and the question this survey asks is what optimization discipline that surface needs. The answer we find is that the field has built an external adaptation layer for LLMs but not the optimizer that should govern it. Most deployments maintain the layer as

patchwork: a failure happens, someone adds a rule, the pile grows, nobody knows which addition helped, and fixes leak into contexts that never validated them.

This article makes five contributions.

1. It states the loop that turns patchwork into optimization in plain language, then as a gated update rule, with the five conditions under which the rule is a mechanism rather than a metaphor (Section III).
2. It introduces a classification of the feedback signal available to that loop, from an exact check to a delayed and sparse reward, and shows that the class fixes what the gate can be, which design-space cells are reachable, and which failure dominates (Section IV).
3. It surveys 256 production and research systems from January 2023 to October 2026, scores each on six substrates with a published rubric and a released score table, and reports where maturity is uneven (Sections V and VI).
4. It places every system that closes both loops in a design space indexed by gate type and reads the empty cells structurally (Section VII), and it reads the convergence of five research families on the same loop as evidence that the architecture is real rather than one group's local optimum (Section VIII).
5. It names, as a four-property audit criterion, the architecture no surveyed system instantiates, namely governed cross-tenant scaffold optimization with versioned lineage, access control, and rollback, and it maps the failure surface that the gap leaves open (Sections IX and X).

The survey runs no experiments. Its evidence is the published behavior of the systems it scores, the released score table, and the audit of every absolute claim against the October-2026 literature (Supplementary Material).

II. WHY ADAPTATION LIVES OUTSIDE THE WEIGHTS

A. BREADTH IN THE WEIGHTS, DEPTH IN THE SCAFFOLD

Frontier training optimizes for breadth. The weights are tuned to generalize across millions of unseen deployments, and the training objective smooths over exactly the things one deployment cares about: local naming conventions, contradictory tenant preferences, rare workflow-specific failures. Reinforcement learning from human feedback reduces output diversity and homogenizes preferred style across users [1], [2], and instruction-tuning data rewards patterns that generalize across the annotator pool rather than patterns specific to any one deployment. The frontier model is therefore trained in a way that systematically disfavors encoding one tenant's idiosyncrasies as preferred behavior. Tiwari et al. [3] reach the same place from the optimization side: in-context adaptation is cheap and rapid but capacity-bounded, while in-weights adaptation is expressive but induces forgetting and loss of plasticity, so a two-timescale system needs a cheap outer layer that absorbs what context can carry.

Production reliability, by contrast, is achieved in depth. Inside one deployment, reliability is determined by what global training had to smooth away. Encoding every deployment in the weights is ruled out three times over: per-tenant fine-tuning multiplies the training, evaluation, and serving lifecycle by the number of deployments; weights ship quarterly at best while the deployment wants daily updates; and overfitting the weights to one tenant degrades adjacent tenants whose preferences contradict. Per-tenant adapters, distilled models, and learned routing each absorb some residual in principle. In practice the scaffold dominates, because it is the one substrate that is cheap, inspectable, reversible, tenant-scoped, and updatable on a daily cadence at once.

B. WHY THE SCAFFOLD IS THE RIGHT ARCHITECTURE, NOT A WORKAROUND: THREE PAPERS

The case that learning belongs outside the weights does not rest on cost alone. It is the conclusion of three papers, of which this is the third. The first, *Beyond Exponential Decay* [4], asks why long LLM outputs do not fail at the rate a uniform per-token error model predicts, and finds that reliability hinges on a sparse set of key decisions, under a tenth of tokens by the available long-context measurement, rather than on uniform per-token accuracy. The second, *The Architecture of Errors* [5], asks what those key-decision failures look like inside a bounded deployment and finds that they cluster into a finite catalogue of recurring modes whose size grows only slowly with the number of observed failures, so a small, targeted budget of interventions covers the residual. Together they say where a deployment's errors are and that they are few enough to fix one at a time. This article asks where the fixes live. The answer the corpus gives is: in persistent artifacts outside the weights, which is where a small catalogue of recurring, tenant-specific modes can be addressed cheaply, inspected, and reversed. Once the loop that edits those artifacts is gated and scoped, the arrangement is not a patch on an inadequate model. It is the architecture that the error structure of the first two papers calls for: a frontier model that generalizes, a local scaffold that specializes, and reliability that comes from governing the specialization.

C. THREE STAGES OF SCAFFOLD MAINTENANCE

Three stages of scaffold maintenance recur across the corpus. They differ in how candidate updates are produced, whether credit assignment is explicit, and whether promotion is bounded by evidence. In the surveyed corpus, production deployments sit at Stage 1 or 2; the loop of Section III is Stage 3.

A patchwork loop becomes Stage 3 once the five conditions of Section III hold. The survey of Sections V to VII asks which deployed systems meet which of them.

D. RELATION TO EXISTING SURVEYS

Table 2 places this survey against the surveys that cover neighboring ground. Existing surveys of self-evolving agents, agent memory, context engineering, skills, and harness design catalogue mechanisms by what they edit and when. None classifies systems by whether a feedback loop is closed, by whether the gate is algorithmic or human, or by the class of signal the gate receives; two carry a governance axis, and

Stage	Update mechanism	Credit assignment	Promotion discipline	Dominant failure
1. Patchwork	Intuition-driven local edits	Implicit, ad hoc	None	Bloat
2. Random artifact search	Many variants tried under an evaluation	Weak; the evaluation is the only signal	Evaluation-gated merge, no failure-to-coordinate tracing	Overfitting to the in-distribution patch
3. Artifact-layer descent	Residual traced to a coordinate, then one edit	Explicit per failure mode	Bounded by evidence radius; reversible	(the discipline this article describes)

TABLE 1. Three stages of scaffold maintenance.

none a cross-organization promotion dimension. This survey classifies on those four axes and audits a named gap against them.

III. THE LOOP: ARTIFACT-LAYER DESCENT

A. THE LOOP IN PLAIN LANGUAGE

The disciplined version of scaffold maintenance is one loop, run again and again over the life of a deployment. It does not react to single failures. As the deployment runs, it logs what went wrong: failed outputs, user corrections, rejected answers, evaluation regressions, tool-call errors. Reflecting over that record, it clusters the failures into recurring modes and takes up the ones that matter most, where a mode earns attention in proportion to how often it recurs and how badly it fails. For such a mode it locates the single scaffold coordinate responsible, which may be a stale memory entry, an under-specified instruction, a missing tool argument, a brittle retrieval rule, or a misrouted agent edge, and it synthesizes one candidate edit to that coordinate. It measures whether the edit lowers the deployment's failure rate on its own recurring tasks, which we call the patch loss, and a gate accepts or rejects the edit on that measurement. An accepted edit stays in the scaffold; one that a later check shows made things worse is pruned or rolled back. Then comes the step most systems skip: the loop decides how far the edit should travel, promoting it to other deployments only as far as the evidence reaches. That limit is the edit's evidence radius. Fig. 1 draws the loop.

B. TWO LOOPS, TWO KNOBS

Consider a concrete case. An engineer notices that a coding agent keeps misnaming a class in one repository. The fix is a one-line rule in that repository's instruction file. The edit is small and its scope is one project. The same engineer could promote the rule to a team-wide template, to the company's core instructions, or to a cross-tenant skill registry. The edit did not change; the scope did. And the gate that should let it through gets stricter at each step, because the loss it is tested against is now an average over more deployments, and a fix that helps one repository can hurt another.

Loop 1 is the local fix: a recurring failure produces a candidate edit, the edit is tested, and an accepted edit changes one deployment's scaffold. Loop 2 is the separate decision of whether that fix should be shared. The two loops turn different knobs. Loop 1 sets how much one accepted edit changes the local scaffold, the analogue of a step size. Loop 2 sets how many deployments the edit applies to, which does not change the step but changes the target: which average failure rate the scaffold is being tuned to reduce. The scopes nest, USER within PROJECT within STACK within TENANT within CORE, and each level has its own loss, the average failure rate over the deployments it contains (Fig. 2). "Cross-tenant promotion" means reaching the TENANT or CORE level. This is why governance is not administrative overhead. It is the regularization that keeps a small fix from being aimed at the wrong target.

C. WHEN THE LOOP IS AN OPTIMIZER

The loop is an optimization process, rather than a description of good practice, when five conditions hold:

1. a measurable patch loss, such as a failure rate, a correction rate, or an evaluation score;
2. captured residuals, so that failures are logged rather than lost;
3. credit assignment, so that a failure can be traced, at least sometimes, to the scaffold coordinate that caused it;
4. a synthesized candidate edit to that coordinate; and
5. a gate that checks whether the edit lowers the loss before the edit is kept or promoted.

Write the patch loss of deployment D as

$$L_D(\theta_s) = \mathbb{E}_{x\sim D}[\ell(\pi(\theta_M, \theta_s, x))]\,,$$

where θ_M are the frontier weights, fixed during deployment, θ_s is the scaffold, a structured object over the six substrate coordinates of Section V, and ℓ is an operational loss. A failure yields no analytic gradient. It yields a candidate edit z_t, proposed by reflection over the accumulated failure history in order of how often and how badly each mode fails. The gate accepts the edit when its noisy estimate of the change in loss clears a margin,

$$G_t(z_t) = 1 \iff \widehat{\Delta}_{z_t} L_D(\theta_s^t) + \lambda\,\Omega(z_t) \le -\tau_t,$$

where $\Omega \ge 0$ penalizes the complexity of the edit and the breadth of its promotion and $\tau_t \ge 0$ is the required margin. An accepted edit is applied; a rejected one leaves the scaffold unchanged. Production systems implement this inequality without writing it: an evaluation blocks a merge on regression, a reviewer rejects a vague rule, a governance system holds a local artifact back until evidence accumulates.

Survey	Scope	Organizing axes	Loop closure	Gate type	Signal class	Governance
Gao et al. [6]	self-evolving agents	what, when, how, where to evolve	partial	partial	no	no
Fang et al. [7]	self-evolving agents	inputs, agent system, environment, optimizers	partial	no	no	no
Zhou et al. [8]	self-evolving coding agents (108 methods)	evolution target, timing, code signals	partial	partial	partial	partial (reversibility)
Ding et al. [9]	persistent-state agents (435 works)	authority, scope, mutability, provenance, recoverability	no	partial	no	yes (provenance, rollback)
Lin et al. [10]	memory security (~90 works)	lifecycle phases × security objectives	no	partial	no	yes (principal scoping, rollbackability)
Guo et al. [11]	agent system and harness design	six runtime responsibilities	partial	partial	partial	partial
Xia et al. [12]	skill-mediated agents (8 systems)	four responsibility layers, ten patterns	partial	no	no	partial
Ding et al. [13]	skill evaluation and evolution	four evolution paradigms × six benchmark classes	partial	partial	partial	partial
Mei et al. [14]	context engineering (1,411 papers)	components × system implementations	no	no	no	no
This survey	256 scaffold-adaptive systems	loop closure, gate type, signal class, governance	yes	yes	yes	yes (four-property audit)

TABLE 2. **Related surveys and the axes they classify by ("partial": discussed but not a classification axis).**

Each loop has a gate, and we name three kinds. A gate is **Auto** when the decision is an algorithmic criterion: an evaluation score, a verifier, a reinforcement-learning threshold, or a language-model judge with a quantified agreement rate. It is **Human** when the decision is reviewer judgment. It is **None** when the loop is absent. These labels index the design space of Section VII.

D. ASSUMPTIONS

The descent reading holds when (i) the frontier weights stay frozen while the scaffold adapts; (ii) each edit is a persistent, inspectable artifact rather than a transient prompt; (iii) the deployment exposes a loss that can be measured; (iv) the proposal process can, at least sometimes, trace a failure to the responsible coordinate; (v) the gate is a noisy estimator of whether an edit helps, not an oracle; and (vi) promotion widens an artifact's scope under a gate rather than averaging comparable updates the way weight-space federated learning does. Under these assumptions Appendix XII states a one-step descent inequality and an informal convergence proposition to a gate-stable scaffold. Where a condition fails, the same picture should be read as an analogy, and the survey marks which systems meet which conditions. Condition (iii) says the loss must be measurable; Section IV asks what measures it.

IV. WHERE THE SIGNAL COMES FROM: FOUR CLASSES OF FEEDBACK

Consider four scaffolds that all run the loop of Section III. A coding agent changes one line in its repository rule file; the test suite runs and reports pass or fail within seconds. A scheduling assistant changes how it phrases a booking request; the appointment either appears on the calendar or it does not, and the system learns which an hour later. A marketing assistant changes the tone rule in its instructions; nobody can say whether the next draft is better, only whether a reviewer prefers it. A research assistant changes how it chooses which direction to pursue; whether the choice was good becomes visible months later, after many other edits have landed. All four produce a candidate edit z_t. They differ in what the gate G_t can know about it.

That difference is the subject of this section. The gate of

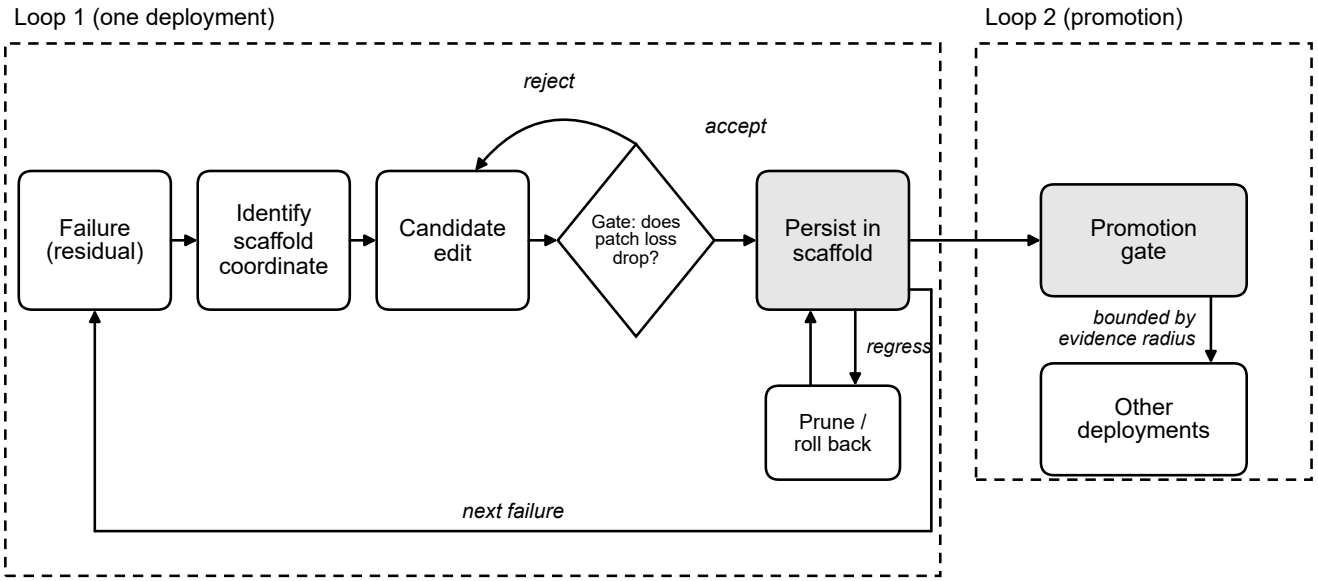


FIGURE 1. **Artifact-layer descent. Loop 1 turns a failure into a gated, persisted edit on one deployment's scaffold; Loop 2 promotes a validated edit to other deployments, only as far as its evidence radius supports.**

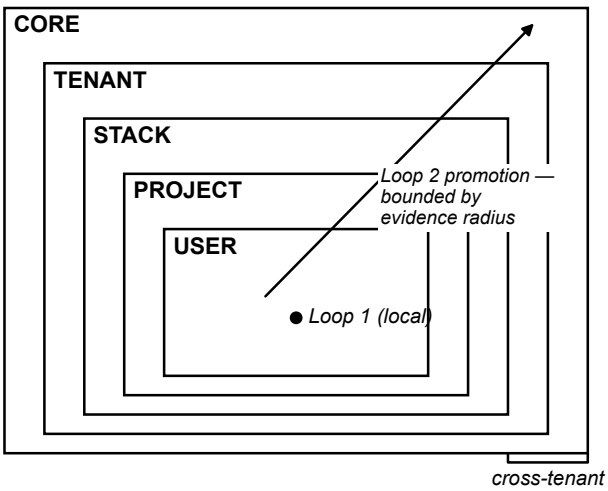


FIGURE 2. **Patch scope hierarchy. Each level's loss averages the deployments it contains. Loop 1 descends a USER or PROJECT loss; Loop 2 promotes upward; cross-tenant promotion means reaching TENANT or CORE.**

Section III accepts an edit when its estimate $\widehat{\Delta}_{z_t} L_D$ of the loss change clears a margin. Condition (iii) of the loop says a patch loss must be measurable; it does not say how. We distinguish four classes of feedback signal by what produces the estimate, and the class turns out to fix three things at once: what the gate can be, which cells of the two-loop design space of Section VII a deployment can reach, and which entry of the failure surface of Section X dominates. Table 3 summarizes; the paragraphs that follow give one example per class from the corpus.

C1, exact. A deterministic checker returns pass or fail, or an exact score, immediately and repeatably. Unit tests, compilers, type checkers, exact-match answer keys, proof checkers, and schema validators are all of this kind. DGM accepts a self-modification only after it passes in a sandbox on coding benchmarks [15]; AlphaEvolve scores candidate programs with a cascade of exact evaluators [16]; SkillOpt gates a skill edit on a held-out exact-match split of a math benchmark [17]. In the formalism the estimator bias ε_t is close to zero, the margin τ_t can be set near zero, and accepting on a single evaluation ($n = 1$, Section VII) is defensible because repeating the check would return the same answer. Two of the five conditions still need work even here: a failing test localizes the output, not the scaffold coordinate that caused it, so credit assignment remains open; and a checker that is also the training target can be gamed. The dominant failures are test overfitting and scaffold bloat, because cheap acceptances accumulate.

C2, observable outcome. The environment or the user emits an outcome after an episode: the appointment is booked, the ticket is closed, the pull request is merged, the tool call returns without error, the retrieved document contains the answer. The signal is real but it arrives one episode late, it is noisy, and it is confounded, because a user can accept a wrong answer and reject a right one. Devin's merged pull requests [18], SAGE's task success on AppWorld [19], SkillRL's

success-rate threshold [20], and PACE's failure signals from a deployed small model [21] are gates of this class. The gate becomes a reinforcement-learning threshold or a held-out task suite; τ_t must be strictly positive, and the noise is absorbed by batching episodes or by requiring agreement across samples (Appendix XIII) rather than by the per-edit check alone. The bottleneck is credit assignment in its hardest form: the outcome says *that* the episode failed, not *why*, so tracing it to one coordinate is an inference the loop must make on its own. This is the class most production deployments live in, and it is also the class in which the signal arrives through the same channel an attacker controls, which is why poisoned memory and tool-output injection (Section X) dominate here.

C3, judged. There is no fact of the matter. An essay, a marketing page, a blog post, a design, a pitch deck, or a code review for style can be scored only by a judge: a human rater, a language-model judge, a rubric, or a comparison against references. SkillForge accepts a skill diff when its judge agrees with human engineers more than nine times in ten [22]; CASCADE consolidates memory under a human-and-agent gate [23]; Anthropic's skills repository promotes a skill when a reviewer approves the pull request. Three things change. The estimator bias ε_t is now the judge's bias, which is bounded only by calibration against people, so assumption (A2) of Appendix XII has to be bought rather than assumed. Residuals are captured only when someone objects, so condition (ii) depends on complaint channels rather than on instrumentation. And the rubric that defines the loss is itself a scaffold coordinate: when the pitch-review worked case of Appendix XII edits the rubric from ''evaluate market size'' to ''require a numeric estimate and a source'', the edit changes the gate that will judge the next edit. We call this the *reflexive gate*. Its dominant failure is Goodhart's law in the form the eval-fragility entry of Section X describes: once the judge is the target, the scaffold learns the judge.

C4, delayed and sparse. The outcome exists but arrives after weeks or months, once per many edits, in a world that has moved on. Research programs, product strategy, and open-ended discovery are of this kind. NanoResearch merges candidate skills by semantic compatibility rather than by a measured loss change [24]; CORAL promotes findings into shared memory on a heartbeat rather than on an evaluation [25]; AlphaEvolve's MAP-Elites archive keeps candidates for their diversity, not their score [16]. There is no per-edit estimate to gate on, so G_t degenerates to proposal pruning plus proxy milestones, and the descent argument applies only to the composition of pruning and whatever gate exists at the horizon (Appendix XIII). Temporal credit assignment across many edits is open. The dominant failure is drift: the scaffold ages faster than the signal that would correct it, and an optimizer in this class cannot be distinguished from random search by its outcomes alone, which is the indistinguishability [26] report for harness optimizers.

The same four classes on the weight side. Reinforcement learning from verifiable rewards trains model weights on C1 signals: a math answer matches or it does not. The effort to push that recipe into writing and other open-ended tasks has had to build C3 gates, and the constructions are worth naming because they are the only gates a C3 scaffold loop can borrow. Rubrics as Rewards scores a response against an instance-specific checklist generated before training and judged by a fixed model [27]; Writing-Zero trains a pairwise generative judge on writing principles and then holds it fixed while the policy learns [28]; reference-anchored judging improves agreement with human labels where no verifier exists [29]; an essay scorer trained against a fixed rubric of binary items reaches the agreement of a single human rater [30]. A second group lets the gate move with the policy: RLPR scores a response by the policy's own probability of a fixed reference, so the scorer co-evolves by construction [31]; Rubric-ARM alternates updates of a rubric generator and a judge before freezing them for policy training [32]; EvoRubrics, SCORE, and RubricEM update rubrics adversarially or from a rubric buffer within training [33]–[35]. Of fourteen such papers we checked, eight hold the rubric or judge fixed during policy optimization and five co-evolve it; five report agreement with human judgment, and none of the co-evolving ones does (Supplementary Material S6). These are weight-space methods and this survey does not score them. The point is narrower: whether the rubric is fixed or co-evolved with the policy is exactly the reflexive-gate question, and a co-evolved rubric is a gate under descent whose calibration nobody has yet measured. On the scaffold side, three systems report judge calibration, SkillForge's agreement with engineers, SkillChain's judge-human kappa, and SkillCoach's trajectory-filter consistency against human gold [22], [36], [37], and none of them calibrates a judge that is itself being edited.

What the class does to the design space. Read the matrix of Section VII by class and its empty cells stop being accidents. Every system in the [Auto $\times$ Auto] cell whose gate is a per-edit estimator runs on a C1 signal or on a simulated C2 environment where the outcome is cheap to observe; the [Auto $\times$ Auto] systems that work in C4 domains gate by pruning, not by an estimate (Claim IV-1). The production-viable cell [Auto $\times$ Human] is populated only by C2 and C3 systems: the human on Loop 2 substitutes for the exact signal that Loop 1 lacks (Claim IV-2). And the weight-side rubric work supplies gate constructions that no scaffold-side system has yet calibrated under the reflexive condition (Claim IV-3). The claims audit in the Supplementary Material checks all three against the rescored corpus; the first two hold without exception and the third is sharpened by the three scaffold-side calibration figures named above.

The class also settles a question Section IX leaves open: whether the missing cross-tenant architecture *should* be filled. The answer is by class rather than by domain. In C1 an automatically gated cross-tenant promotion of a test-validated code skill is low-risk, and the held-out receiving-side evaluation that Section VI finds absent is cheap to add. In C2 the corner should be filled only with that receiving-side evaluation, because the sending side's outcome signal does

Class	Signal	Gate G_t becomes	Conditions of Sec. III satisfied	Reachable cells (Sec. VII)	Dominant failure (Sec. X)	Corpus examples
C1 exact	deterministic pass/fail or exact score, immediate	surrogate verifier; exact eval cascade; $\tau_t \approx 0$	(i)(ii)(iv)(v) directly; (iii) credit assignment open	[Auto × Auto], [Auto × Human], [None × Auto]; only class where auto-gated Loop 2 needs no receiving-side human	test overfitting; scaffold bloat	DGM, AlphaEvolve kernels, SkillOpt on math, Voyager
C2 observable outcome	success/failure event after one episode; noisy, confounded	RL threshold; held-out task suite; batching or agreement filter; $\tau_t > 0$	(i) noisy; (ii) only if logged; (iii) hardest; (iv)(v)	[Auto × Auto] in simulators; [Auto × Human] in production	patch overfitting; poisoned memory; tool-output injection	Devin, Sierra, SAGE, SkillRL, PACE
C3 judged	judge, rubric, rating, reference comparison; no ground truth	calibrated LLM-judge or reviewer; the rubric is itself a coordinate (reflexive gate)	(i) by proxy; (ii) via complaints; (iii) via rubric decomposition; (iv); (v) proxy	[Auto × Human] naturally; [Auto × Auto] only with calibration; [Human × Human] for high stakes (empty)	Goodhart / eval fragility; rubric accretion	SkillForge, CASCADE, skills-repository review
C4 delayed, sparse	outcome at a long horizon, once per many edits	proposal pruning (semantic merge, archive selection) plus proxy milestones; no per-edit gate	(i) only at horizon; (ii) rarely as a failure event; (iii) temporal, open; (iv); (v) replaced by novelty or taste	[Auto × Auto] through archive selection; otherwise [None × Auto] or human on both loops	drift; indistinguishable from random search	NanoResearch, CORAL, AlphaEvolve archive

TABLE 3. **The four feedback-signal classes and what each fixes.**

not transfer. In C3 a human gate on Loop 2 is a design feature, not a defect, and governance means lineage and rollback around a human decision. In C4 property (a) of Section IX cannot be met, so what remains of governance is lineage and rollback without an automatic gate. Safety-critical domains, which Section IX names as the case for human gates, are C3 or C4 domains with high stakes; the class, not the domain, is what carries the argument.

V. SUBSTRATES, CORPUS, AND SCORING

A. SIX SUBSTRATES

A deployment's scaffold is composed of six substrates. They are the coordinates of θ_s, and each can be edited independently of the others.

The 0 to 5 rubric is the same on every substrate: 0, ephemeral; 1, persistent and local; 2, persistent with tool attachment but no feedback; 3, multi-scope or feedback-driven but without automated promotion; 4, automated feedback updates the scaffold, so one loop is closed; 5, two-loop versioned promotion with governance, so both loops are closed. Other partitions are possible, for instance separating evaluation suites from governance; the argument of Section III is invariant to the partition so long as the coordinates stay independently editable.

B. HOW THE CORPUS WAS ASSEMBLED

The corpus was built in two passes with one rubric. The first pass (January 2023 to May 2026) searched arXiv, vendor documentation, and public repositories for systems described as self-improving, scaffolded, or agentic, and kept those that wrap a foundation model with at least one persistent artifact updated from deployment signal. The second pass (June to October 2026) ran query families for self-evolving skills and skill lifecycle, harness and context optimization, agent memory governance, prompt-as-code and evaluation-gated promotion, federated or multi-tenant skill sharing, and multi-agent topology evolution, and followed citations of the systems the first pass had identified as closest to the gap of Section IX. Every system admitted in either pass was scored from its own method section or documentation, never from a secondary summary. The first pass scored 130 systems; the second added 126, for 256 unique systems and 269 substrate-level rows. The cutoff is October 2026.

Two filters structure the corpus (Fig. 3). The patch-local discriminator admits a system to the core corpus when it scores at least 3 on at least one substrate, that is, when some artifact persists across scope or is updated from feedback; systems that carry the labels but fail it are kept as a contrast corpus so the boundary is auditable rather than rhetorical. A stricter composite two-loop audit then asks whether Loop 1

Substrate	What persists	Score-5 mark
S1 Instructions	Project, team, or organization rules attached to a workspace	Versioned, two-loop, auto-gated promotion
S2 Skills	Procedural artifacts (code or text) loaded on demand	Skill bank self-modifies on failure, test-gated, versioned, promoted
S3 Memory	Cross-session experience (episodic, semantic, procedural)	Provenance, versioning, and correction pathways
S4 Tools	Vetted action surface and context schemas	Domain-curated bundle is the product; evaluation-gated tool versions
S5 Orchestration	Multi-role graph and routing	Population or policy updates from an outer-loop signal
S6 Governance	Versioning, promotion, audit, rollback	Dev-to-staging-to-production with evaluation thresholds, audit log, and rollback

TABLE 4. The six scaffold substrates and the mark a system must reach to score 5.

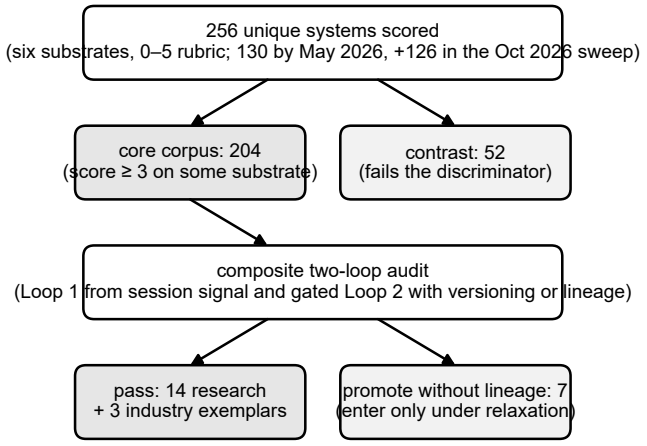


FIGURE 3. Corpus flow. Every system is scored on the six substrates; the discriminator splits core from contrast; the composite two-loop audit selects the systems placed in the design space of Section VII.

modifies a persisted artifact from session signal and Loop 2 has an explicit cross-context promotion mechanism with versioning or lineage; only systems passing both enter the design space of Section VII. Of the 256 systems, 204 pass the discriminator and 52 form the contrast corpus; 14 research systems and 3 industry exemplars pass the composite audit, and 7 systems added in 2026 promote across contexts without versioning or lineage and enter only under the relaxations below. Industry systems evaluated from vendor documentation rather than from a paper are flagged as such wherever they appear.

Exclusions. Systems whose only update mechanism is weight-level fine-tuning or reinforcement learning are model-side and belong to the self-evolution literature this survey positions against; systems with no public evidence of deployment or release are excluded.

C. EVIDENCE AND SCORING DISCIPLINE

Each system carries one or more evidence tiers: T1 peer-reviewed publication; T2 arXiv preprint; T3 production claim with metrics; T4 open-source artifact; T5 industry blog or interview. T1 and T4 carry the most weight in cluster-level statements; T3 is admissible when the claim is concrete (a named benchmark or integration target); T5 only when corroborated by a higher tier. Production claims are never pooled with peer-reviewed evidence when a cluster-level statistic is computed. Scoring is single-rater: one author scored every system, half-scores were used only when one criterion was clearly met and another partially, and ambiguous cases were rounded down. The complete score table, with a one-line justification per row, is released as Supplementary Material S2 so that every classification can be re-derived. The counts reported in Section VI move under two natural relaxations of the composite criterion, counting any of versioning, lineage, or rollback as Loop-2 governance, or admitting updates that persist only optimizer state as Loop 1; the qualitative findings survive both (Supplementary Material S2).

VI. FINDINGS: SUBSTRATE MATURITY AND FOUR CLUSTERS

A. MATURITY IS UNEVEN, AND THE 2026 ADDITIONS DID NOT CHANGE THE CEILING

Table 5 aggregates the corpus by substrate. The column *n* counts scored rows on that substrate, *Mean* is the average score, and *Score-5 systems* are those that reach the substrate's top mark in Table 4.

The rows behind the table read as follows. On S1, rules persist across IDEs and agents (Claude Code, Cursor, AGENTS.md, Windsurf, Continue.dev) and adoption is measured at a fifth to a quarter of coding-agent projects [38], [39], yet context files raise inference cost by about a fifth without raising task success in a controlled study [40]. On S2, 31 of the 2026 additions are skill systems, almost all closing Loop 1 under an automatic gate, and eleven treat retirement as a gated operation [41]–[43]; the public skill repositories, by contrast, are human-merged with one deprecation in 254

Substrate	*n*	Mean	Score-5 systems	Survey finding
S1 Instructions	15	3.07	(none)	Rules persist across IDEs and agents; none closes both loops; ceiling 4
S2 Skills	50	3.34	SAGE, COSPLAY	Most active research substrate; Loop 1 routinely automated; production repositories human-merged
S3 Memory	50	3.06	Cuadros et al.	Lineage now shipped in products; enforced correction still absent
S4 Tools	20	3.65	MCP, Harvey, Hippocratic AI	Most mature production substrate; registries govern admission, not evaluation
S5 Orchestration	37	3.08	(none)	Topologies evolve under evaluation; one structurally gated instance; no two-loop system
S6 Governance	45	3.02	Braintrust, Vellum, LangSmith Hub, AGENTS.md/AAIF	Mature prompts-as-code; 2026 registries add lineage and rollback; no inner loop
Integrated	52	3.81	NanoResearch, AutoAgent, SkillRL	High-water mark; research systems without enterprise governance

TABLE 5. Substrate maturity across the 256-system corpus (all scored rows; May 2026 corpus plus October 2026 sweep). No system added in 2026 scores 5 on any substrate.

edits [44]. On S3, lineage has arrived in production memory, in Letta's git-backed Context Repositories [45], Mem0's per-memory history [46], and Zep's bitemporal facts [47], while five production backends still return revoked facts [48]; the 2026 research splits into write-gating and provenance on one side [49]–[51] and verifier-gated failure loops on the other [52]–[54], and no system carries both. On S4, MCP registries split into catalogs (the official registry: namespaces and version records, no scanning, no private servers) and control planes with scanning, access control, and quarantine [55]–[57]; none has evaluation-gated promotion. On S5, topologies now evolve under evaluation [58]–[60], TOCOMAS alone retains a topology change only when it satisfies a structural invariant [61], and Evo2Team promotes routing skills across teams with target-side confirmation [62]. On S6, the 2026 registries bring lineage and rollback [63]–[65], and every one of them has Loop 1 = None.

Three things stand out. First, the field has converged on the six substrates: every mature system has a recognizable instance of each, under different names. Second, no system matures all six at once. Research systems learn fast and lack enterprise governance; production systems govern well and learn slowly; open standards solve artifact portability but not adaptation. Third, the 2026 sweep added 126 systems, 92 of its 102 core systems close Loop 1 under an automatic gate, and not one of them reaches the top mark on any substrate: the local loop has become routine and the governed second loop has not moved. The maturity gradient is not a quirk of the corpus. It tracks which constraints each kind of system was built to satisfy.

B. FIVE ABSENCES, READ BY SIGNAL CLASS

Five absences fall out of the discriminator, and each sits in a particular feedback-signal class (Section IV).

- **Failure-triggered memory is rare in production** (C2). Deployed memory systems record preferences and successful facts, not residuals; the one field report of anti-recurrence stores written from dead-ends is a single self-reported deployment [66].
- **Cross-customer promotion is nearly absent** (all classes). ShareMem, EvolveNet, XRepoSkill, and RASO promote across users, clients, or repositories inside one operator [67]–[70]; no system promotes across customers or organizations with evidence.

- **Versioned memory has arrived; enforced correction has not** (C2, C3). Git-backed and bitemporal memory ship in products, and five backends still return revoked facts [48].
- **Scaffold-level curriculum is unspecified in nearly every system** (C2). Which sessions inform the next edit is left to the order they arrive in; Task-CoEvolve's variance-weighted task sampling and AutoSaddler's mini-batch failure signals are the exceptions [71], [72].
- **Receiving-side evaluation at promotion is rare** (C1, C2). It now exists in ShareMem (evaluation tasks belong to a different user), XRepoSkill (verification in at least two repositories), Evo2Team (target-side confirmation, which still misses four of twenty-two transfers on new graphs), and VACE (candidate evaluated with the receiving model held fixed) [73]; EvolveNet has none, and the federated optimization framework that proposes a receiving-client gate has not implemented it [74]. This is the missing piece that would catch a scaffold over-fitting one deployment before it propagates to another.

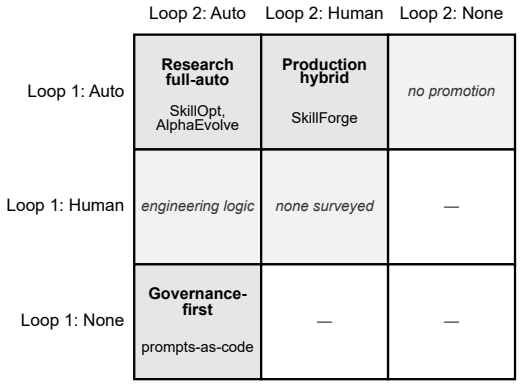


FIGURE 4. The two-loop design space. Three cells hold every system that passes the composite audit; the human-authored cells are occupied only by registries and curation workflows that have no session-signal loop. No populated system adds governed cross-tenant promotion.

C. FOUR CLUSTERS

Four archetypal clusters recur across the core corpus. They differ in where on the patchwork-to-descent gradient each sits and in which loop is automated.

Research full-auto systems show that the architecture works under fully algorithmic gates. NanoResearch co-evolves skills, memory, and policy with a semantic-merge orchestrator [24]; SkillOpt runs held-out gating, a rejected-edit buffer, and a bounded textual learning rate on a markdown skill (Section VIII); AlphaEvolve pairs a cascade evaluator with a diversity archive and reports production gains at Google [16]; DGM modifies its own code under sandboxed tests [15]. The 2026 entries move the cluster's frontier from a single deployment to a federation of them, but every member still lacks access control and audited promotion.

Production hybrid systems pair an algorithmic Loop-1 gate with a reviewer on Loop 2. SkillForge routes judge-passed skill diffs to support engineers, with the judge agreeing with people more than nine times in ten [22]; Spotify optimizes prompts and planning automatically and launches only behind an online A/B test [75]; SkillChain applies judge-monotone edits and then a human reflection gate, with judge-human agreement reported as a kappa between 0.72 and 0.91 [36]. The human on Loop 2 is doing the work that the missing exact signal cannot.

Governance-first systems automate Loop 2 alone. The prompts-as-code platforms and the 2026 registries [63]–[65] have immutable versions, evaluation thresholds, and rollback; they lack a Loop 1 because candidates are authored by engineers rather than triggered by session feedback. This is MLOps for scaffolds, distinct from MLOps for weights.

Open standards and registries converge across vendors on format and attachment. Claude Agent Skills, Cursor Rules, GitHub Copilot instructions, Windsurf, Continue.dev, Zed, and Replit all keep a git-tracked markdown artifact attached to a workspace with an MCP-style tool layer [39], [76]. Convergence across vendors is independent evidence, complementary to the research-side convergence of Section VIII, that the architecture is real rather than any one vendor's local optimum. Two measured facts temper the optimism: context files raise cost without raising success in a controlled study [40], and the runtime state of autonomous loops sits outside version control in the repositories that run them [77].

VII. THE TWO-LOOP DESIGN SPACE

Systems that pass the composite audit sit in a 3×3 matrix indexed by the gate type on each loop (Fig. 4); Supplementary Material S4 lists the named membership of every cell.

Three cells hold every system that passes the composite audit. [Auto × Auto] holds the research full-auto systems, twelve from the May corpus with machine-evaluated criteria on both loops. [Auto × Human] is the production-viable cell: Loop 1 fires at machine speed under an algorithmic criterion and Loop 2 waits for a person. [None × Auto] holds the governance-first platforms with evaluation-gated promotion and no failure-triggered candidate generation. The 2026 sweep did not add a cell: its seven cross-context promoters land in [Auto × Auto] without versioning or lineage and so enter the strict audit only under relaxation, and its one production loop lands in [Auto × Human].

The other cells are not empty, but they are occupied only by systems that fail the composite audit, and the pattern of occupancy is itself a finding. [Human × Auto] remains empty: nobody authors candidates by hand and promotes them automatically, because manual authorship offers automated promotion little leverage. [Human × Human] is occupied by MCP registries and a community hub [55]–[57], [78], where a person registers an artifact and a person approves its use; the field study of five public skill repositories finds the same shape in practice, with every substantive edit human-merged and one deprecation in 254 edits [44]. [Human × None] is the curation cell: an agent drafts, a person fact-

Cluster	Representative systems	Loop 1 / Loop 2	What the cluster shows
Research full-auto	NanoResearch, SkillOpt, SkillRL, AlphaEvolve, DGM, ADAS, Voyager; in 2026 EvolveNet, Evo2Team, XRepoSkill	Auto / Auto	Both loops can run under algorithmic gates and benchmark gains compound; enterprise governance is uniformly absent
Production hybrid	SkillForge, CASCADE, Sierra Agent OS 2.0, Cognition Devin; in 2026 Spotify's recommendation loop [75], SkillChain [36]	Auto / Human	An algorithmic Loop-1 gate with a reviewer on Loop 2 is the production-viable cell; the 2026 entries add online A/B gating and reported judge agreement
Governance-first	Braintrust, Vellum, LangSmith Hub; in 2026 EDDOps, Agentic Configuration Management, Governed Capability Evolution	None / Auto or Human	Evaluation-gated promotion, lineage, and rollback exist; candidate generation is manual and no inner loop fires from session signal
Open standards and registries	AGENTS.md, MCP, Anthropic Agent Skills, Cursor Rules; in 2026 the MCP control planes	Mixed, or Human / Human	Cross-vendor convergence on artifact format and tool attachment; registries govern admission and access but do not evaluate

TABLE 6. Four clusters of the core corpus.

checks, and the result is frozen [79]. These are single-loop by construction. Read by signal class, the occupancy is what Section IV predicts: automatic gates on both loops appear only where the signal is exact or cheaply observable, and the human cells fill where it is not.

Two cross-cutting findings sharpen the picture. *Evidence is now batched before acceptance.* In the May corpus nearly every automatic gate fired on a single evaluation, absorbing estimator noise through the per-edit margin alone. By mid-2026 statistical acceptance is common: a paired per-item test that commits an edit only when its wins are reliable [80], a merge accepted only when the held-out items it fixes outnumber those it breaks [68], three heterogeneous critics that must all pass [81], and mini-batch failure signals with validation-based selection [72]. The agreement form of batching, where k independent proposals must converge, shrinks the false-accept rate as p^k rather than shrinking variance as $1/k$, and the two should not be conflated (Appendix XIII). *Multi-layer hierarchies remain rare.* AlphaEvolve is still the only system that combines a cascade evaluator with a diversity archive; the 2026 systems that keep archives use them for lineage, not for staged evaluation.

VIII. CONVERGENT RESEARCH FAMILIES

The loop of Section III is not a prescription the field has yet to adopt. It is the limit that research is converging on from several directions at once, under different names and for different substrates, and the convergence is evidence that the architecture is real rather than one group's local optimum.

The clearest single instance is SkillOpt [17]. It treats one markdown skill document as the trainable external state of a frozen agent and edits it with explicit optimizer controls: rollout and reflection batches, add/delete/replace edits, a bounded textual learning rate with cosine decay, a held-out selection gate that accepts only a strictly better score, a rejected-edit buffer kept as negative feedback, and an epoch-wise slow update. Across six benchmarks, seven target models, and three harnesses it is best or tied on every cell its authors evaluate, and it reports transfer across models, harnesses, and benchmarks that reads as an empirical measurement of evidence radius rather than as a governance mechanism. Its own follow-up, SkillOpt-Lite [82], removes the buffer, the slow update, and the schedule, keeps the held-out gate and a consensus filter across rollouts, and matches or beats it: the gate and a consensus form of credit assignment are load-bearing, and the rest are stabilizers.

Five research families instantiate complementary parts of the loop (Table 7). The *reflective-evolution* family proposes edits by reflecting over trajectories [83] and supplies the two-timescale motivation [3]. *Reflective context learning* [84] is the most direct independent formalization: it observes that credit assignment, overfitting, forgetting, and high-variance signals persist whether the learned object lies in parameter space or context space, and its generalization penalty is the complexity term Ω of Section III. The *harness-optimization* family targets orchestration rather than skills (Meta-Harness [85]; AHE [86]; MOSS [87]; CANTANTE [88]; HARNESSEVO [89]) and names credit assignment as the bottleneck; by October 2026 it is the largest family in the corpus, with roughly two hundred papers citing its founding entries and a dozen scored here. The *lifecycle* family shows that hygiene is a precondition for descent: LLM-authored skills add nothing over no skill until retirement and a bounded active set are in place [90], and eleven 2026 systems make retirement a gated operation [41]–[43], [91]. The *transfer* family instruments Loop 2: measuring evidence

radius one task at a time [92], pooling evidence across parallel agents [93], promoting through shared memory on a heartbeat [25], consolidating across a deployment's own episodes [21], and, in 2026, federating skill patches across clients with an auditable decision log [94] or across deployments with a held-out merge gate and round rollback [68].

Read against the loop and against the four properties of Section IX, four claims are well supported.

1. The scaffold-versus-weights decomposition is independently confirmed: GEPA, Meta-Harness, MOSS, RCL, PACE, and the 2026 harness papers all begin from the observation that weights and the external artifact layer are distinct adaptation surfaces with different cost, reversibility, and scope. A system that updates both keeps them as separate levers [95].
2. Credit assignment is the central engineering bottleneck. AHE, CANTANTE, Meta-Harness, HARNESSEVO, SkillsVote, and the harness-optimizer evaluation of Ong et al. [26] all identify attributing a system-level failure to one scaffold coordinate as what separates descent from trial-and-error, and it remains unsolved in the general multi-substrate case.
3. Lifecycle hygiene is a precondition for descent, not an alternative to it. A dirty artifact pool starves the gate of signal; the 2026 retirement mechanisms are the field's answer, and the one negative result cuts deeper: once a contaminated skill has descendants, rolling it back recovers little, so gating must happen before commit [81].
4. No surveyed system combines automated local adaptation with governed cross-organization promotion. FederatedSkill and EvolveNet narrow the gap on different properties and close it on none.

IX. THE MISSING ARCHITECTURE: GOVERNED CROSS-TENANT SCAFFOLD OPTIMIZATION

Four properties define an architecture that no surveyed system instantiates:

- (a) **auto-gated Loop 1**: the commit criterion is an algorithmic threshold;
- (b) **cross-organization promotion**: Loop 2 routes artifacts between organizations (we use *cross-tenant* and *cross-organizational* interchangeably);
- (c) **versioned lineage with access control**: promoted artifacts carry versions, audit trails, and role-based access policies;
- (d) **rollback**: a bad promotion can be reverted without redeploy.

The closest approaches partition the requirements, and the October-2026 sweep sharpened the partition without closing it. FederatedSkill has (a), the strongest partial (b) in the corpus (privacy-preserving aggregation of skill patches across heterogeneous clients that sit under one operator), and partial (c) through an auditable per-modification decision log [94]. EvolveNet has (a), a held-out merge gate at the aggregator, and (d) in the form of a rollback to the previous shared harness after a failed round, with no lineage, no access control, and clients that are benchmark partitions rather than organizations [68]. SkillForge has (a) and (b) within one enterprise with partial (c) [22]. The governance-first registries have (c) without access control and (d) with evidence [63]–[65], and the MCP control planes have access control without evaluation-gated promotion [55]. Rollback now appears inside single deployments, as edit-level reversal [42], [96] and as an admission test that rejects edits which cannot be undone [97], and inside one federated round; no system rolls back a promotion that has crossed an organization boundary, because none makes such a promotion. Supplementary Material S4 gives the per-system enumeration.

In machine-learning terms, what is missing is a cross-tenant aggregator with an automated evaluation gate at the scaffold layer. It would carry five parts: per-tenant artifact pools, each tagged with provenance; an aggregation step across tenants that respects tenant isolation; a held-out evaluation at the receiving tenant that blocks a promotion whenever it does not lower that tenant's patch loss; versioned lineage with access control; and a rollback envelope on every promoted artifact. The federated-averaging analogy [98], [99] is useful only in the limit where artifacts are mergeable update representations; in surveyed practice Loop 2 is selection and merge, and the nearest systems aggregate program edits with a language model, not by averaging.

Whether the architecture *should* be built is a question the feedback-signal class answers (Section IV). Where the signal is exact, an automatically gated cross-tenant promotion of a test-validated skill is low-risk and the receiving-side evaluation is cheap. Where the signal is an observable outcome, the corner should be filled only with that receiving-side evaluation, because the sending side's outcome does not transfer; Evo2Team's four failed transfers out of twenty-two after target-side confirmation show that even a receiving-side check is necessary rather than sufficient [62]. Where the signal is judged, a human gate on Loop 2 is a design feature, and governance means lineage and rollback around a human decision. Where the signal is delayed, property (a) cannot be met and what remains is lineage and rollback without an automatic gate. Naming the four properties converts a vague gap into an audit criterion that any future system can be measured against; naming the signal class says, for a given deployment, which of the four it should expect to meet.

X. THE FAILURE SURFACE

Patchwork scaffold maintenance introduces failure modes that weight-only systems do not have. Each is also a missing piece of the optimizer, and each dominates in a particular feedback-signal class (Table 8). This section surveys the documented attacks and degradations; it proposes no new threat model.

Two rows deserve a sentence beyond the table. Bloat is a signal-quality problem as well as a latency problem: Ratchet reports LLM-authored skills at no gain over a no-skill baseline against a double-digit gain for human-curated ones, and attributes the gap to lifecycle management, which in the

Family	Representative systems	What it instantiates	What it leaves open
Reflective evolution	GEPA; Tiwari et al.	proposal by reflection over trajectory evidence; two-timescale motivation	persistent artifact; bounded step; held-out gate
Reflective context	RCL	the learned object in context space; generalization penalty as Ω	cross-deployment Loop 2
Harness optimization	Meta-Harness; AHE; MOSS; CANTANTE; HARNESSEVO; HarnessForge; Growing Harness	credit assignment over orchestration; source-level edits; slot attribution	governed Loop 2; access control; rollback
Lifecycle and multi-objective	Ratchet; MOCHA; SkillsVote; Library Drift; SkillSmith; ASSAY	hygiene as precondition; constraint frontier as the shape of Ω; gated retirement	gradient-analogue closure; retirement of a promoted artifact's descendants
Transfer and federation	ICT; Combee; CORAL; PACE; FederatedSkill; EvolveNet; Evo2Team; XRepoSkill	empirical evidence radius; cross-client aggregation; receiving-side confirmation	cross-organization gate, lineage, rollback

TABLE 7. **Five research families converging on artifact-layer descent, with the part of the loop each instantiates.**

Failure	What has been measured	Architectural response	Dominant class
Scaffold bloat	Instruction-following accuracy falls and latency grows sharply as instruction count scales [100]; LLM-authored skills add nothing over no skill until lifecycle hygiene is applied [90]	Complexity penalty Ω in the gate; pruning and expiry as first-class gated operations	C1, C3
Poisoned memory	High attack success against standard retrieval-augmented generation [101]; memory control-flow attacks across major agent frameworks [102]	Provenance on every entry; write gating; a rollback envelope [103], [104]	C2
Injection via tools and MCP	A tool's output or a connected server can smuggle instructions into context; one approved MCP configuration could later be modified without re-approval (CVE-2025-54136) [105]	Narrow tool permissions; tool bindings pass the same promotion gate as any other coordinate	C2
Evaluation fragility	Access asymmetries alone produce large relative gains that reflect exposure to the test [106]; harness optimizers cannot be told from trial-and-error by downstream score alone [26]	Held-out evaluations scoped within evidence radius, weighted toward recurring modes, re-read as the target drifts, and checked against an update's attribution	C3 (primary), C1
Coordination failures	Fourteen failure modes in multi-agent systems, dominated by role confusion and context loss [107]	Orchestration topology treated as a scaffold coordinate under the same gate and rollback discipline	C2
Patch overfitting	A scaffold tuned to one deployment over-fits it by construction (Appendix XII)	Evidence-radius discipline at promotion; receiving-side held-out evaluation	C2
Plasticity loss	Context rot, instruction conflict, memory pollution, tool-version drift	Expiry and rollback as explicit loop operations rather than retrospective hygiene	C4

TABLE 8. **The failure surface of patchwork scaffold maintenance, the measured evidence, the architectural response, and the feedback-signal class in which each failure dominates.**

vocabulary of Section III means that residual capture and mode discovery cannot work on a dirty artifact pool [90]. And evaluation fragility compounds because the evaluation suite is itself a patch-local artifact rather than a fixed instrument. It is optimized against, so once a metric becomes the target it is gamed. It is read non-independently, because the gate scores path-dependent agent trajectories whose readings do not compose like ordinary test-set statistics. It goes stale,

because the suite adapts on the same cadence as the scaffold it gates. And because failures concentrate in a finite catalogue of recurring modes, a suite that samples tasks at random under-weights exactly the families that drive the failure rate. The response in the table follows from those four properties: held-out, scoped, mode-weighted, re-read, and checked against attribution rather than only against score.

The failure surface is real, quantified by 2024 to 2026 work, and largely unmitigated in production; a survey of persistent-state agents coded from 435 works reaches the same verdict, finding the literature concentrated on accumulating and retrieving state rather than on governing, recovering, or relinquishing it [9]. Every row names a piece of the optimizer that current systems do not yet have.

XI. OPEN PROBLEMS AND CONCLUSION

Five open problems follow from the survey, one per feedback-signal class and one for the architecture.

1. **Exact signals: credit assignment from a failing check to a scaffold coordinate.** A test tells the loop that an output failed, not which instruction, skill, or tool binding caused it. Slot attribution [89], intervention-based attribution [59], and contrastive multi-agent attribution [88] each solve one substrate's case; a general method is the bottleneck named by every harness-optimization paper.
2. **Observable outcomes: receiving-side evaluation before promotion.** Four systems now evaluate at the receiving side and one of them still misses a fifth of its transfers; the gate that a federated scaffold optimizer needs is a held-out test at the receiving tenant, proposed but not yet built [74].
3. **Judged signals: calibrating a gate that is itself being edited.** Fourteen weight-side rubric-reward papers include five that co-evolve the rubric with the policy, and none of those five reports agreement with human judgment; on the scaffold side three systems report judge calibration and none calibrates a judge under edit. The reflexive gate of Section IV has no measured instance.
4. **Delayed signals: telling descent from search.** Where the outcome arrives after many edits, an optimizer's gains cannot be distinguished from trial-and-error by outcome alone [26]; evaluation of harness optimizers against their attribution claims rather than their scores is the open method.
5. **The governed cross-tenant corner.** Properties (a) through (d) of Section IX, with the receiving-side gate of problem 2 as the piece that would make cross-organization promotion safe in the two classes where it should be automatic.

The field has built an external adaptation layer for language models but not the optimizer that should govern it. The reason the layer belongs where it is was established by the two papers this article completes: reliability over long outputs hinges on a sparse set of key decisions, and inside a bounded deployment those decisions fail in a small, recurring catalogue of modes that a targeted budget of fixes can cover. The place to spend that budget is a persistent, inspectable, reversible artifact outside the weights, and once the loop that edits it is gated and scoped, the arrangement is not a patch on an inadequate model but the architecture that the error structure calls for. The survey shows how far practice has come toward that loop. The local loop closes automatically in most research systems and in a growing number of production ones; evidence is batched before acceptance; retirement and expiry have become first-class; lineage has reached production memory. What has not moved is the second loop. No surveyed system promotes a validated edit across an organization boundary under an automatic gate with lineage, access control, and rollback, and the signal-class analysis says this is not one gap but four, each with its own reachable answer. The frontier model generalizes. The local scaffold specializes. Reliability comes from governing that specialization, and the instruments that judge each edit are themselves part of the moving scaffold, so the same evidence-radius and lifecycle discipline the optimizer needs must extend to the gates that judge it.

XII. APPENDIX: FORMAL STATEMENT OF ARTIFACT-LAYER DESCENT

This appendix states the setup behind Section III, a one-step descent inequality under a noisy gate, an informal convergence proposition, the optimization-analogy correspondence, and four worked cases. No new theorem is claimed; the proposition is the ordinary target of local stochastic optimization under noisy estimators, restated for a discrete, versioned parameter.

A. SETUP

Let the frontier model have fixed parameters θ_M and let the scaffold be a structured object $\theta_s \in \Theta_s = \Theta_1 \times \cdots \times \Theta_6$ over the six substrate coordinates of Section V; Θ_s is discrete, symbolic, and versioned. For a deployment D, $L_D(\theta_s) = \mathbb{E}_{x\sim D}[\ell(\pi(\theta_M, \theta_s, x))]$. A candidate edit $z_t \sim Q_t(z \mid H_t, \theta_s^t)$ is drawn from a proposal process conditioned on the accumulated history H_t of failures, corrections, rejected outputs, evaluation regressions, and prior accepted edits. Q_t is not uniform: for a mode m that recurs with frequency f_m and carries per-occurrence loss s_m, its expected contribution to the failure rate is $f_m s_m$, so high-frequency or high-severity modes are proposed first. The finite directional difference is $\Delta_z L_D(\theta_s) = L_D(\theta_s \oplus z) - L_D(\theta_s)$; the gate sees only a noisy estimate $\widehat{\Delta}_z L_D$. The update is

$$\theta_s^{t+1} = \theta_s^t \oplus z_t \text{ if } G_t(z_t) = 1, \quad \theta_s^{t+1} = \theta_s^t \text{ otherwise.}$$

B. DESCENT INEQUALITY

Assume (A1) positive descent bias on accepted edits, $\mathbb{E}[\widehat{\Delta}_{z_t} L_D \mid G_t = 1, \theta_s^t] \le -\gamma_t - \tau_t$ for some $\gamma_t > 0$, and (A2) bounded estimator bias on accepted edits, $|\mathbb{E}[\Delta_{z_t} L_D - \widehat{\Delta}_{z_t} L_D \mid G_t = 1, \theta_s^t]| \le \varepsilon_t$. With $p_t = \Pr(G_t = 1 \mid \theta_s^t)$,

$$\mathbb{E}[L_D(\theta_s^{t+1}) \mid \theta_s^t] \le L_D(\theta_s^t) - p_t(\gamma_t + \tau_t) + p_t\,\varepsilon_t.$$

Per-step expected improvement is the gate's margin minus its estimator bias, weighted by acceptance probability. In the

feedback-signal classes of Section IV, ε_t is near zero for an exact check (C1), bounded by batching for an observable outcome (C2), equal to the judge's bias and bounded only by calibration for a judged signal (C3), and unavailable per edit for a delayed signal (C4).

C. CONVERGENCE (INFORMAL)

Add (A3) proposal coverage, that at every non-gate-stable scaffold Q_t places probability at least $q > 0$ on an admissible edit with $\mathbb{E}[\Delta_z L_D] \leq -\delta$; (A4) effective compactness of the reachable orbit; and (A5) persistent acceptance with summable bias, $\sum_t p_t(\gamma_t + \tau_t) = \infty$ and $\sum_t p_t \varepsilon_t < \infty$; and let $0 \leq L_D \leq B$. Then $L_D(\theta_s^t)$ is a non-negative supermartingale up to summable noise, it converges almost surely, and the limit set of $\{\theta_s^t\}$ is gate-stable: no admissible edit clears the gate with negative expected directional loss in the limit. This is convergence of the loss and stability of the gate decision, not convergence to an optimum; a globally suboptimal gate-stable scaffold is consistent with the result, because Q_t may never propose the improving edit or $\widehat{\Delta}$ may be biased to reject it. In a non-stationary deployment ($D = D_t$) the same mechanism tracks a moving optimum rather than converging once, which is why recency weighting, pruning, expiry, and rollback are the scaffold-level analogues of continual-learning machinery rather than optional hygiene.

D. CORRESPONDENCE

Credit assignment here is attribution, not backpropagation. Scaffold systems do not differentiate through instruction files; they trace an output error to an upstream coordinate, manually or with a judge or verifier. The descent argument needs only that this attribution succeeds with non-zero probability per failure, assumption (iv) of Section III.

E. WORKED CASES

Retrieval. Failures recur because retrieved context omits the decisive clause. Attribution traces the error to the retrieval coordinate: chunking, query rewriting, metadata filter, embedding model, reranker, or context-packing rule. The candidate edit tightens chunk boundaries or adds a metadata constraint, and the gate checks that recall and answer accuracy improve on the patch evaluation without degrading adjacent cases. Signal class C1 or C2.

Pitch review. The system repeatedly overrates decks that claim a large market without numbers. The failure cluster points at an under-specified rubric coordinate. The edit changes "evaluate market size" into "require a numeric estimate and a source; if absent, cap the score". The gate runs the revised rubric against held-out decks with human scores. The edit changed the gate that will judge the next edit: the reflexive gate of Section IV, class C3.

Tool use. Calls fail because optional parameters are underspecified. The responsible coordinate may be the tool schema, the call policy, or the clarification rule. The edit adds required fields or a pre-call uncertainty check; the gate checks that tool-call failures fall without raising refusals or latency. Class C2.

Memory. A stale customer preference is repeatedly retrieved and causes bad decisions. The responsible coordinate is a memory entry together with its provenance, priority, and expiry policy. The edit corrects, expires, or version-tags the entry; the gate checks that future behavior improves without erasing useful history. Class C2, with the correction itself observable only when a later outcome reveals it.

XIII. APPENDIX: GATE FAMILIES AND FEEDBACK-SIGNAL CLASSES

Section III collapses every production gate into one per-edit test G_t on an estimate of the loss change. Surveyed systems instantiate that test in several distinct ways, and some of them are better read as proposal pruning that precedes the per-edit gate than as estimators of $\widehat{\Delta} L_D$. Table 10 lists the families, the slot each occupies, representative systems, and the feedback-signal class (Section IV) each family serves.

The first five families populate G_t directly and inherit the descent guarantee of Appendix XII under its assumptions. Semantic merge and archive selection are not estimators of $\widehat{\Delta} L_D$; they constrain the proposal distribution, and the descent argument applies to the composition of pruning and gate. Multi-sample agreement is listed because it is the correct analogue of "wait for k pieces of evidence before accepting an edit", which is a different operation from mini-batch averaging and should not be conflated with it. Read down the signal-class column: the only families available to a judged (C3) deployment are the two whose estimator is a judge, and the only families available to a delayed (C4) deployment are the two that do not estimate the loss change at all. That is the structural content of Section IV.

ACKNOWLEDGMENT

The authors used an AI assistant (Claude, Anthropic) to help search the literature, check citations, and edit prose; all claims, scores, and classifications were reviewed and approved by the authors. Earlier versions of this argument appeared as an arXiv preprint and as a non-archival workshop submission; this article extends them with the feedback-signal classification of Section IV, a rescored corpus, and a claims audit.

REFERENCES

[1] R. Kirk, I. Mediratta, C. Nalmpantis *et al.*, "Understanding the effects of RLHF on LLM generalisation and diversity," in *ICLR 2024*, 2024.
[2] V. Padmakumar and H. He, "Does writing with language models reduce content diversity?" *arXiv preprint*, 2023.
[3] R. Tiwari, K. Sareen, L. A. Agrawal, J. E. Gonzalez, M. Zaharia, K. Keutzer, I. S. Dhillon, R. Agarwal, and D. Khatri, "Learning, fast and slow: Towards LLMs that adapt continually," *arXiv preprint*, 2026.
[4] M. L. Arbuzov, S. Bei, Z. Dong, D. Kalaev, and A. A. Shvets, "Beyond exponential decay: Rethinking error accumulation in large language models," *arXiv preprint*, 2025.
[5] M. L. Arbuzov, L. Mosbacker, S. Bei, Z. Dong, D. Kalaev, and A. Shvets, "The architecture of errors: From universal impossibility to patch-local LLM reliability," *arXiv preprint arXiv:2605.30628*, 2026.

Optimization concept	Scaffold analogue
Parameter vector	Instructions, skills, memories, tools, routing graphs, evaluation gates, governance rules
Loss	Failure, user correction, evaluation regression, retrieval miss, tool error, schema violation
Gradient estimate	Credit assignment identifying the coordinate responsible for a failure
Step	Accepted scaffold edit; step size is edit magnitude under $\oplus$
Aggregated update	Several recurring failures combined before an update, by averaging or by agreement across k proposals
Update radius	Promotion scope: which loss is descended (USER, PROJECT, STACK, TENANT, CORE)
Validation loss	Held-out patch evaluation before merge
Regularization	Complexity penalty $\Omega(z)$, bloat control, cross-patch regression checks
Overfitting	Scaffold improves on this deployment and degrades elsewhere
Rollback	Reverting a harmful accepted update

TABLE 9. **Optimization concepts and their scaffold analogues.**

Gate family	What it estimates	Slot in Section III	Signal class	Representative systems
Surrogate verifier	Pass or fail from tests, a symbolic checker, or an exact match	Per-edit gate G_t	C1	DGM (sandboxed code), SkillOpt (held-out exact match), Voyager (game-state check: C2)
Evaluation cascade	$\widehat{\Delta}L_D$ on a held-out task suite, cheap-to-expensive evaluators	Per-edit gate G_t	C1 when the suite is exact; C2 when it is a task suite	AlphaEvolve, Braintrust, Vellum, LangSmith Hub
RL threshold	Reward improvement against a deployment threshold	Per-edit gate G_t	C2	SkillRL, SAGE, EvolveR, PACE
LLM judge with calibration	Pairwise or rubric score, calibrated against human agreement	Per-edit gate G_t	C3	SkillForge (Loop 1), CASCADE (consolidation gate)
Reviewer judgment	Human credit assignment by pull-request review or expert sign-off	Per-edit gate G_t, human-instantiated	C3 (and C2 when the reviewer has outcome data)	SkillForge (Loop 2), Sierra Agent OS 2.0, Anthropic skills repository
Semantic merge	Whether a candidate duplicates or conflicts with existing artifacts	Proposal pruning before G_t	C4 (also a pre-filter in C2, C3)	NanoResearch orchestrator, FederatedSkill server agent
Archive selection	Whether a candidate fills an unfilled cell in a behavior-diversity archive	Proposal pruning before G_t	C4	AlphaEvolve (MAP-Elites), ADAS
Multi-sample agreement	Whether k independent proposals or evaluations agree; false-accept rate decays as p^k rather than variance as $1/k$	Aggregation filter on the estimate	C2, C3 noise absorption	Under-explored; SkillOpt-Lite's consensus attribute mining is the nearest instance

TABLE 10. **Gate families, their slot in the formalism, and the feedback-signal class each serves.**

[6] H.-a. Gao, J. Geng, W. Hua, M. Hu, X. Juan *et al.*, "A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence," *Transactions on Machine Learning Research*, 2026, arXiv:2507.21046v4, 16 Jan 2026.

[7] J. Fang, Y. Peng, X. Zhang, Y. Wang, X. Yi, G. Zhang, Y. Xu, B. Wu, S. Liu, Z. Li, Z. Ren, N. Aletras, X. Wang, H. Zhou, and Z. Meng, "A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems," *arXiv preprint arXiv:2508.07407*, Aug. 2025.

[8] H. Zhou, H. Hu, T. Luo, Y. Shang, C. Fang, Z. Chen, L. Xiao, and Q. Zhang, "Self-evolving coding agents," *arXiv preprint arXiv:2608.03392*, Aug. 2026.

[9] T. Ding, A. Nannapaneni, B. Liu, and L. Zhang, "Always-on agents: A survey of persistent memory, state, and governance in LLM agents," *arXiv preprint arXiv:2606.30306*, 2026.

[10] Z. Lin, X. Hao, R. Fu, S. Cui, K. Chen, C. Li, Z. Li, and F. Xiong, "A survey on long-term memory security in LLM agents: Attacks, defenses, and governance across the memory lifecycle," in *Proceedings of EMNLP*

2026, 2026, arXiv:2604.16548v3, 22 Sep 2026.

[11] J. Guo, Z. Hao, C. Wang, C. Fan, T. Luo, H. Li, Y. Gao, H. Mei, J. Peng, R. Xu, M. Dong, H. Wu, M. Zheng, K. Han, S. Wang, C. Xu, and Y. Wang, "From question answering to task completion: A survey on agent system and harness design," *arXiv preprint arXiv:2606.20683*, Jun. 2026.

[12] B. Xia, L. Zhu, Z. Xing, Q. Lu, D. Sejdinovic, and X. Xu, "Harnessing agent skills: Architectural patterns and a reference architecture for skill-mediated LLM agents," *arXiv preprint arXiv:2606.20631*, May 2026.

[13] K. Ding, Y. Zhou, C. Jin, F. Tong, M. Zhou, and D. N. Metaxas, "Agent skill evaluation and evolution: Frameworks and benchmarks," *arXiv preprint arXiv:2606.11435*, Jun. 2026.

[14] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li, D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, and S. Liu, "A survey of context engineering for large language models," *arXiv preprint arXiv:2507.13334*, Jul. 2025.

[15] J. Zhang *et al.*, "Darwin Gödel machine: Open-ended evolution of self-improving agents," *arXiv preprint*, 2025.

[16] A. Novikov *et al.*, "AlphaEvolve: A coding agent for scientific and algorithmic discovery," arXiv preprint arXiv:2506.13131, 2025.

[17] Y. Yang, Z. Gong, W. Huang, Q. Yang, Z. Zhou, Z. Huang, Y. Li, X. Gao, Q. Dai, B. Liu, K. Qiu, Y. Yang, D. Chen, X. Yang, and C. Luo, "SkillOpt: Executive strategy for self-evolving agent skills," arXiv preprint arXiv:2605.23904, 2026. [Online]. Available: https://microsoft.github.io/SkillOpt

[18] Cognition, "Devin's 2025 performance review: Learnings from 18 months of agents at work," https://cognition.com/blog/devin-annual-performance-review-2025, 2025.

[19] J. Wang *et al.*, "Reinforcement learning for self-improving agent with skill library," *arXiv preprint*, 2025.

[20] P. Xia *et al.*, "SkillRL: Evolving agents via recursive skill-augmented reinforcement learning," *arXiv preprint*, 2026.

[21] C. Ling, P. Chen, A. Guan, J. Qu, S. A. Akbar, M. Gopinathan, and E. Cornejo, "PACE: Two-timescale self-evolution for small language model agents," *arXiv preprint*, 2026.

[22] X. Liu *et al.*, "SkillForge: Forging domain-specific, self-evolving agent skills in cloud technical support," *arXiv preprint*, 2026, accepted at ACM SIGIR 2026 Industry Track.

[23] X. Huang *et al.*, "CASCADE: Cumulative agentic skill creation through autonomous development and evolution," *arXiv preprint*, 2025.

[24] J. Xu *et al.*, "NanoResearch: Co-evolving skills, memory, and policy for personalized research automation," *arXiv preprint*, 2026.

[25] A. Qu, H. Zheng, Z. Zhou, Y. Yan, Y. Tang, S. Y. Ong, F. Hong, K. Zhou, C. Jiang, M. Kong, J. Zhu, X. Jiang, S. Li, C. Wu, B. K. H. Low, J. Zhao, and P. P. Liang, "CORAL: Towards autonomous multi-agent evolution for open-ended discovery," *arXiv preprint*, 2026.

[26] K. T.-i. Ong, M. Kang, D. Choi, J. Cho, S. Kim, S. Lim, G. Jang, M. Oh, B. Jeong, S. Kim, T. Kwon, and J. Yeo, "Towards direct evaluation of harness optimizers via priority ranking," *arXiv preprint*, 2026.

[27] A. Gunjal, A. Wang, E. Lau, V. Nath, Y. He, B. Liu, and S. Hendryx, "Rubrics as rewards: Reinforcement learning beyond verifiable domains," *arXiv preprint arXiv:2507.17746*, 2025.

[28] R. Jia, Y. Yang, Y. Gai, K. Luo, S. Huang, J. Lin, X. Jiang, and G. Jiang, "Writing-zero: Bridge the gap between non-verifiable tasks and verifiable rewards," *arXiv preprint arXiv:2506.00103*, 2025.

[29] K. Shi, Y. Liu, P. Wang, A. R. Fabbri, S. Joty, and A. Cohan, "References improve LLM alignment in non-verifiable domains," *arXiv preprint arXiv:2602.16802*, Feb. 2026.

[30] X. Jin, J. Zhang, T. Cao, and B. Yang, "Beyond score prediction: LLM-based essay scoring and feedback generation via reinforcement learning with rubric rewards," *arXiv preprint arXiv:2607.19219*, Jul. 2026.

[31] T. Yu, B. Ji, S. Wang, S. Yao, Z. Wang, G. Cui, L. Yuan, N. Ding, Y. Yao, Z. Liu, M. Sun, and T.-S. Chua, "RLPR: Extrapolating RLVR to general domains without verifiers," *arXiv preprint arXiv:2506.18254*, Jun. 2025.

[32] R. Xu, T. Liu, Z. Dong, T. Yu, I. Hong, C. Yang, L. Zhang, T. Zhao, and H. Wang, "Alternating reinforcement learning for rubric-based reward modeling in non-verifiable LLM post-training," *arXiv preprint arXiv:2602.01511*, 2026.

[33] H. Ding, B. Huang, Y. Fang, W. Liao, Z. Li, J. Zhang, Z. Wu, J. Zhao, and Y. Wang, "EvoRubrics: Dynamic rubrics as rewards via adversarial co-evolution for LLM reinforcement learning," *arXiv preprint arXiv:2606.23038*, Jun. 2026.

[34] H. Zhu, C. Cai, Y. Song, X. Chen, S. Han, and Y. Guo, "Self-evolving deep research via joint generation and evaluation," *arXiv preprint arXiv:2606.04507*, Jun. 2026.

[35] G. Li, B. Dalvi Mishra, Z. Wang, J. Yan, Y. Chen, C.-L. Li, L. T. Le, R. Han, G. Lee, H. Tong, C.-Y. Lee, and T. Pfister, "RubricEM: Meta-RL with rubric-guided policy decomposition beyond verifiable rewards," *arXiv preprint arXiv:2605.10899*, May 2026.

[36] Y. Hu, M. Xu, H. Guo, Y. Song, X. Zhu, and B. Zheng, "Skillchain: Closing the loop on skill evolution for image-based e-commerce ai assistants," *arXiv preprint arXiv:2606.12984*, 2026.

[37] J. Zhu, K. Mao, Y. Guo, D. He, S. Xu, S. Gu, and Y. Yue, "Skillcoach: Self-evolving rubrics for evaluating and enhancing agentic skill-use," *arXiv preprint arXiv:2607.01874*, 2026.

[38] R. Robbes, T. Matricon, T. Degueule, A. Hora, and S. Zacchiroli, "Agentic much? adoption of coding agents on github," *arXiv preprint arXiv:2601.18341*, 2026.

[39] M. Galster, S. Mohsenimofidi, J. L. Lulla, M. A. Abubakar, C. Treude, and S. Baltes, "Harness engineering for agentic ai coding tools: An exploratory study," *arXiv preprint arXiv:2602.14690*, 2026.

[40] T. Gloaguen, N. Mündler-Sasahara, M. N. Müller, V. Raychev, and M. Vechev, "Evaluating agents.md: Are repository-level context files helpful for coding agents?" *arXiv preprint arXiv:2602.11988*, 2026.

[41] X. Zhang, Y. Cui, G. Wang, Z. Li, W. Qiu, B. Zhu, and P. He, "Library drift: Diagnosing and fixing a silent failure mode in self-evolving llm skill libraries," *arXiv preprint arXiv:2605.19576*, 2026.

[42] Y. Wei, Z. Huang, S. Lu, J. Qian, Q. Wang, C. Wu, and L. He, "Skillsmith: Co-evolving skills and tools for self-improving agent systems," *arXiv preprint arXiv:2606.01314*, 2026.

[43] H. Pu, X. Song, and L. Zhao, "Skillops: Managing llm agent skill libraries as self-maintaining software ecosystems," *arXiv preprint arXiv:2605.13716*, 2026.

[44] C. Shen and E. Hruschka, "Who maintains agent skills? A longitudinal study of human-governed, AI-assisted skill maintenance," *arXiv preprint arXiv:2609.05677*, 2026.

[45] Letta, "MemFS; memory & dreaming; introducing context repositories: Git-based memory for coding agents," https://docs.letta.com/letta-code/memfs, https://docs.letta.com/letta-agent/memory, https://www.letta.com/blog/context-repositories/, 2026.

[46] Mem0, "Mem0 platform: Memory history, feedback, and changelog," https://docs.mem0.ai/api-reference/memory/history-memory, https://docs.mem0.ai/api-reference/memory/feedback, https://docs.mem0.ai/changelog, 2026.

[47] Zep, "Zep documentation: Facts; february 2026 deprecation wave; changelog," https://help.getzep.com/facts, https://help.getzep.com/february-2026-deprecation-wave, https://help.getzep.com/changelog/2026/9/24, 2026.

[48] Y. T. Shen, K. Toyoda, and A. Leung, "Revoked but still authoritative: An empirical study of revocation enforcement in agent-memory systems," *arXiv preprint arXiv:2609.08258*, Sep. 2026.

[49] Z. Wang, "TOKI: A bitemporal operator algebra for contradiction resolution in LLM-agent persistent memory," *arXiv preprint arXiv:2606.06240*, Jun. 2026.

[50] J. Xu, Y. Xiao, W. Shao, H. Liu, and X. Li, "Memory provenance laundering in LLM agents: A non-amplification firewall for persistent memory," *arXiv preprint arXiv:2607.29167*, Jul. 2026.

[51] Y. Margalit, N. Cohen-Inger, E. Avram, R. Taig, and O. Margalit, "Governed shared memory for multi-agent LLM systems," *arXiv preprint arXiv:2606.24535*, Jun. 2026.

[52] Z. Yu, Y. Wu, Z. Yin, K. Chen, Z. Zhao, M. Wang, S. Yan, and L. Yang, "Recursive experiential–working memory evolution for long-horizon agent harnesses," *arXiv preprint arXiv:2608.24876*, Aug. 2026.

[53] H. Wang, G. Dong, H. Liang, Z. Zhang, J. Luo, C. Liu, C. Xue, and H. Tang, "MemGuard: Persisting verifier signals for LLM-agent memory governance," *arXiv preprint arXiv:2608.21867*, Aug. 2026.

[54] X. Sun, Q. Wang, H. Li, and L. Chen, "Verifiable memory: Learning unified memory management with local and global verifiers for large language model agents," *arXiv preprint arXiv:2608.03137*, Aug. 2026.

[55] agentic-community, "Mcp gateway & registry," https://github.com/agentic-community/mcp-gateway-registry, 2026.

[56] Microsoft, "Inventory and discover mcp servers in your api center," https://learn.microsoft.com/en-us/azure/api-center/register-discover-mcp-server, 2026.

[57] TrueFoundry, "Mcp gateway authentication and security," https://www.truefoundry.com/docs/ai-gateway/mcp-gateway-auth-security, 2026.

[58] Y. Cheng, C. Liu, H. Yu, H. Tan, T. Wu, W. Jin, J. Ran, K. Wang, X. Li, H. Bai, G. Chesi, and N. Wong, "Optimas: Automatically optimize multi-agent system," *arXiv preprint arXiv:2608.21918*, 2026.

[59] J. Chu, J. Seo, J. Cho, J. Na, Y. Xiong, Y. Kim, and H. J. Kim, "Agentgrad: Intervention-guided prompt optimization for multi agent systems," *arXiv preprint arXiv:2609.08572*, 2026.

[60] M. Chen, C. Lv, G. Zhang, H. Chang, and S. Zhou, "Harnessforge: Joint harness and policy evolution for adaptive agent systems," *arXiv preprint arXiv:2606.01779*, 2026.

[61] S. Zhao, R. Kong, Z. Zhang, L. Shen, X. He, X. Zhang, and Q. Zhang, "Topological coherence for self-evolving multi-agent systems," *arXiv preprint arXiv:2609.37953*, 2026.

[62] R. Wang and J. Cui, "Evo2team: When do evolved skills transfer? from selection to deployment," *arXiv preprint arXiv:2609.34135*, 2026.

[63] R. Kang and V. Wang, "Registry-governed agent lifecycle: Completing EDDOps with evaluation-driven registration, promotion, and retirement on AWS AgentCore," *arXiv preprint arXiv:2607.00345*, Jul. 2026.

[64] A. Quessada-Vial, "Agentic configuration management (ACM): A reference configuration model for governed agentic systems," *arXiv preprint arXiv:2608.11166*, Aug. 2026.

[65] X. Qin, S. Luan, J. See, Z. Boukhers, C. Yang, and Z. Li, "Governed capability evolution: Lifecycle-time compatibility checking and rollback for AI-component-based systems, with a proof-of-concept evaluation on embodied agents," *arXiv preprint arXiv:2604.08059*, Apr. 2026.

[66] M. Helwig, "Memory as infrastructure: Reliability engineering for persistent agent memory in months-long LLM-assisted development," *arXiv preprint arXiv:2609.05510*, Aug. 2026.

[67] J. Hu, H. Zhao, Q. Jia, D. Chen, T. Zhao, S. Duan, and G. Liu, "Learning from others, acting for you: Cross-user memory sharing for llm agents," *arXiv preprint arXiv:2609.32511*, 2026.

[68] J. Nie, Y. Zhang, Q. Cai, Y.-m. Cheung, X. Tian, and B. Han, "Evolvenet: Collaborative harness evolution for agent self-improvement," *arXiv preprint arXiv:2608.04968*, 2026.

[69] Y. Guo, H. Zhou, J. Zhang, Y. Zhang, Y. Liu, Q. Chen, Q. Lin, H. Dong, J. M. Zhang, and Z. Chen, "Xreposkill: Learning transferable skills for software engineering agents," *arXiv preprint arXiv:2609.36807*, 2026.

[70] J. Chu, J. S. Lee, J. Park, D. Ko, J. Na, S. Lee, T. Lee, M. Yoon, M. Joo, Y. Xiong, and H. J. Kim, "Retrieval-augmented skill optimization via cross-harness adaptation," *arXiv preprint arXiv:2609.38024*, 2026.

[71] A. Miyai, K. Aizawa, and T. Yamasaki, "Task-coevolve: Efficient harness optimization via adaptive validation task selection," *arXiv preprint arXiv:2608.20169*, 2026.

[72] S. Park, W. Kim, R. Tan, J. Zhang, W.-S. Han, P. Gao, C. Park, Y. Yao, R. Fu, E. Nallipogu, Q. Lin, S. Rajmohan, and D. Zhang, "Autosaddler: Automatic harness optimization with durable updates from agent execution traces," *arXiv preprint arXiv:2608.23041*, 2026.

[73] J. Qi, Y. He, J. Liu, Q. Huang, S. Hu, Z. Dang, G. Chen, R. Yang, W. Jiang, Y. Liu, T. Lyu, and F. Li, "Vace: Validation-gated alternating co-evolution of agent models and harnesses," *arXiv preprint arXiv:2609.37105*, 2026.

[74] Q. Yang, Z. Kou, X. Zhang, D.-D. Wu, H. Gu, J. Guo, Y. Liu, D. Jiang, and Q. Xu, "Federated agent optimization," *arXiv preprint arXiv:2610.01195*, 2026.

[75] E. Palumbo, A. Tamborrino, V. Ode, B. Lacker, A. Casas Escoda, J. Hopple, M. Better, J. Leoni, H. Galvão, H. Bouchard, M. Lalmas, J. L. Redondo García, A. Abebe, A. Clifton, A. Blomberg, H. Lindström, D. Doro, and C. Doig Cardet, "Bootstrapping conversational recommendation agents at spotify: Synthetic data generation and self-improvement loops," *arXiv preprint arXiv:2609.30297*, 2026.

[76] P. Barbaste, T. Darrigol, G. Vu, and T. Wiltberger, "Harness engineering: Anatomy, architecture, and evolution of coding agents – a source-code study of eleven systems," *arXiv preprint arXiv:2609.00006*, 2026.

[77] J. L. Lulla, V. Nersesyan, S. Mohsenimofidi, C. Treude, and S. Baltes, "Loop engineering: Building blocks, adoption, and impact," *arXiv preprint arXiv:2608.21884*, 2026.

[78] Z. Jiang and B. Dong, "Buildrix: An open platform for sharing and benchmarking agentic ai skills in building engineering," *arXiv preprint arXiv:2606.25139*, 2026.

[79] T. Q. Nguyen, S. Desai, R. H. Anwar, F. Shaik, V. Suryanarayanan, and V. Chowdhary, "VerificAgent: Domain-specific memory verification for scalable oversight of aligned computer-use agents," *arXiv preprint arXiv:2506.02539*, Jun. 2025.

[80] Y. Wang, L. Xia, R. Liu, Z. Zhang, H. Wu, Y. Yang, J. He, Y. Guo, and K. Lei, "Sage: A statistical acceptance gate for self-evolving agents," *arXiv preprint arXiv:2609.36043*, 2026.

[81] L. Shang, M. Xu, Y. Sun, T. Xia, L. Hu, L. Xu, and N. Zheng, "When self-evolution backfires: Pre-commit gating against skill contamination in llm agents," *arXiv preprint arXiv:2608.05810*, 2026.

[82] Y. Shen, B. Li, and X. Zhang, "SkillOpt-Lite: Better and faster agent self-evolution via one line of vibe," *arXiv preprint arXiv:2607.03451*, 2026.

[83] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab, "GEPA: Reflective prompt evolution can outperform reinforcement learning," *arXiv preprint*, 2025.

[84] N. Vassilyev, W. Berrios, R. Zhang, B. Han, D. Kiela, and S. Mehri, "Reflective context learning: Studying the optimization primitives of context space," *arXiv preprint*, 2026.

[85] Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab, and C. Finn, "Meta-Harness: End-to-end optimization of model harnesses," *arXiv preprint*, 2026.

[86] J. Lin, S. Liu, C. Pan, L. Lin, S. Dou, Z. Xi, X. Huang, H. Yan, Z. Han, T. Gui, and Y.-G. Jiang, "Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses," *arXiv preprint*, 2026.

[87] Q. Cai, Y. Zhang, X. Jia, H. Zheng, W. Xue, J. Song, X. Tian, and Y. Guo, "MOSS: Self-evolution through source-level rewriting in autonomous agent systems," *arXiv preprint*, 2026.

[88] T. Zehle, "CANTANTE: Optimizing agentic systems via contrastive credit attribution," *arXiv preprint*, 2026.

[89] M. Nguyen, W. C. Tan, N. A. Hassan, A. Raman, L. H. Lim, and A. F. Razak, "Where does harness-optimization value live? Localized gains and the budget-splitting trap in self-evolving LLM agents," *arXiv preprint arXiv:2609.02889*, 2026.

[90] X. Zhang, Y. Cui, G. Wang, Z. Li, W. Qiu, B. Zhu, and P. He, "Ratchet: A minimal hygiene recipe for self-evolving LLM agents," *arXiv preprint*, 2026.

[91] Y. Wang, Y. Zhou, Y. Liang, C. Zhang, F. Liu, J. Zhou, and H. Yao, "Not all skills help: Measuring and repairing agent knowledge," *arXiv preprint arXiv:2606.15390*, 2026.

[92] Y. Shalev, Z. Ding, and M. Jamnik, "Training language agents to learn from experience," *arXiv preprint*, 2026.

[93] H. Li, R. He, Q. Zhang, C. Ji, Q. Mang, X. Chen, L. A. Agrawal, W.-L. Liao, E. Yang, A. Cheung, J. Zou, K. Olukotun, I. Stoica, and J. E. Gonzalez, "Combee: Scaling prompt learning for self-improving language model agents," *arXiv preprint*, 2026.

[94] J. Yang, G. Yao, Y. Zhang, R. R. Kompella, G. Liu, and S. Chang, "FederatedSkill: Federated learning for agentic skill evolution," *arXiv preprint arXiv:2606.03143*, 2026.

[95] P. Hebbar, Y. Manawat, S. Verboomen, A. Ivanova, S. Palanimalai, K. Bhatia, and V. Baskaran, "Sia: Self improving ai with harness & weight updates," *arXiv preprint arXiv:2605.27276*, 2026.

[96] H. Gao, H. Chen, C. Wang, S. Guo, L. Pang, Z. Liu, H. Shen, and X. Cheng, "Skillaudit: Ground-truth-free skill evolution via paired trajectory auditing," *arXiv preprint arXiv:2606.14239*, 2026.

[97] T. Sah, D. Sah, H. Jain, and T. Sah, "Evoundo: Recoverability-constrained self-evolution for llm agent harnesses," *arXiv preprint arXiv:2608.28363*, 2026.

[98] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 54, 2017, pp. 1273–1282.

[99] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *International Conference on Learning Representations (ICLR)*, 2018, arXiv:1710.06963.

[100] D. Jaroslawicz *et al.*, "How many instructions can LLMs follow at once?" *arXiv preprint*, 2025.

[101] W. Zou *et al.*, "PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models," *arXiv preprint*, 2024, accepted at USENIX Security 2025.

[102] Z. Xu *et al.*, "From storage to steering: Memory control flow attacks on LLM agents," *arXiv preprint*, 2026.

[103] Y. Su, W. Xu, C. Zuo, and E. Bertino, "ChronoMem: Version control and semantic rollback for large language model agent memory," *arXiv preprint arXiv:2607.27773*, 2026.

[104] C. Yu, Y. Wang, J. Zhang, Y. Duan, M. Zheng, Z. Wu, K. Shi, and T. Cai, "From faulty memories to corrected actions: Dependency-guided rollback repair for memory-augmented agents," *arXiv preprint arXiv:2608.10502*, 2026.

[105] Check Point Research, "MCPoison: Cursor IDE's MCP vulnerability (CVE-2025-54136)," https://research.checkpoint.com/2025/cursor-vulnerability-mcpoison/, 2025.
[106] S. Singh *et al.*, "The leaderboard illusion," *arXiv preprint*, 2025.
[107] M. Cemri *et al.*, "Why do multi-agent LLM systems fail?" *arXiv preprint*, 2025.

MIKHAIL L. ARBUZOV TODO(bio).

LEE MOSBACKER TODO(bio).

SISONG BEI TODO(bio).

ZIWEI DONG TODO(bio).

DMITRI KALAEV TODO(bio).

ALEXEY SHVETS TODO(bio).